Helper Functions

```
import math

def find_dice_combinations(num_dice, faces, target_sum):
    # dp[i][j] stores the number of ways to get sum j using i dice
    dp = [[0] * (target_sum + 1) for _ in range(num_dice + 1)]

    # Base case: 1 way to get sum 0 with 0 dice (empty set)
    dp[0][0] = 1

    # Fill the dp table
    for i in range(1, num_dice + 1):  # For each die
        for j in range(1, target_sum + 1):  # For each possible sum
            for face_val in range(1, min(faces + 1, j + 1)): # For each face va
                if j - face_val >= 0:
                    dp[i][j] += dp[i - 1][j - face_val]
    return dp[num_dice][target_sum]

def nCr(n, r):
    if r < 0 or r > n:
        return 0
    if r == 0 or r == n:
        return 1
    if r > n // 2:
        r = n - r

    # Using dynamic programming for binomial coefficient (Pascal's Triangle app
    C = [[0] * (r + 1) for _ in range(n + 1)]

    for i in range(n + 1):
        for j in range(min(i, r) + 1):
            if j == 0 or j == i:
                C[i][j] = 1
            else:
                C[i][j] = C[i-1][j-1] + C[i-1][j]
    return C[n][r]

def assembly_line_scheduling(a, t, e, x):
    # n = number of stations
    n = len(a[0])

    # dp[0][j] stores minimum time to reach station j on line 0
    # dp[1][j] stores minimum time to reach station j on line 1
    dp = [[0] * n for _ in range(2)]

    # Time taken to reach first station in line 0 or line 1
```

```
    dp[0][0] = e[0] + a[0][0]
    dp[1][0] = e[1] + a[1][0]

    # Fill dp table for remaining stations
    for j in range(1, n):
        # For line 0
        dp[0][j] = min(
            dp[0][j - 1] + a[0][j],            # Stay on line 0
            dp[1][j - 1] + t[1][j - 1] + a[0][j] # Switch from line 1 to line 0
        )

        # For line 1
        dp[1][j] = min(
            dp[1][j - 1] + a[1][j],            # Stay on line 1
            dp[0][j - 1] + t[0][j - 1] + a[1][j] # Switch from line 0 to line 1
        )

    # Add exit times and return the minimum of the two last stations
    return min(dp[0][n - 1] + x[0], dp[1][n - 1] + x[1])
```

Dice Throw Problem Solutions

Question 1: Dice Throw Problem

```
num_dice = 3
faces = 6
target_score = 8
result = find_dice_combinations(num_dice, faces, target_score)
print(f"Number of ways = {result}")
```

```
Number of ways = 21
```

Question 2: Board Game Tournament

```
num_dice = 2
faces = 6
target_score = 7
result = find_dice_combinations(num_dice, faces, target_score)
print(f"Number of ways = {result}")
```

```
Number of ways = 6
```

Question 3: Warehouse Robot Navigation

```
num_dice = 4
faces = 4
target_distance = 10
result = find_dice_combinations(num_dice, faces, target_distance)
print(f"Number of ways = {result}")
```

```
Number of ways = 44
```

Question 4: Casino Reward Calculation

```
num_dice = 3
faces = 6
target_sum = 12
result = find_dice_combinations(num_dice, faces, target_sum)
print(f"Number of ways = {result}")
```

```
Number of ways = 25
```

Question 5: Quiz Competition Scoring

```
num_dice = 5
faces = 6
target_score = 15
result = find_dice_combinations(num_dice, faces, target_score)
print(f"Number of ways = {result}")
```

```
Number of ways = 651
```

Question 6: Sports Prediction System

```
num_dice = 3
faces = 5
target_score = 9
result = find_dice_combinations(num_dice, faces, target_score)
print(f"Number of ways = {result}")
```

```
Number of ways = 19
```

Question 7: Lottery Simulation System

```
num_dice = 2
faces = 10
target_sum = 11
```

```
result = find_dice_combinations(num_dice, faces, target_sum)
print(f"Number of ways = {result}")
```

```
Number of ways = 10
```

Question 8: General Dice Sum (N dice, M faces)

```
num_dice = 2
faces = 6
target_sum = 7
result = find_dice_combinations(num_dice, faces, target_sum)
print(f"Number of ways = {result}")
```

```
Number of ways = 6
```

Question 9: General Dice Sum (multiple dice)

```
num_dice = 3
faces = 6
target_sum = 8
result = find_dice_combinations(num_dice, faces, target_sum)
print(f"Number of ways = {result}")
```

```
Number of ways = 21
```

Question 10: Count possible outcomes that produce a specified sum

```
num_dice = 4
faces = 6
target_sum = 10
result = find_dice_combinations(num_dice, faces, target_sum)
print(f"Number of ways = {result}")
```

```
Number of ways = 80
```

Question 11: Find the number of possible dice combinations that yield the required score

```
num_dice = 2
faces = 4
target_sum = 5
result = find_dice_combinations(num_dice, faces, target_sum)
print(f"Number of ways = {result}")
```

```
Number of ways = 4
```

Binomial Coefficient Solutions

Question 1: Determine the number of ways to obtain the target sum using dice throws.

```
num_dice = 3
faces = 4
target_sum = 7
result = find_dice_combinations(num_dice, faces, target_sum)
print(f"Number of ways = {result}")
```

```
Number of ways = 12
```

Question 2: Calculate the Binomial Coefficient C(n,r)

```
n = 5
r = 2
result = nCr(n, r)
print(f"C({n},{r}) = {result}")
```

```
C(5,2) = 10
```

Question 3: Find the number of ways to select r items from n items

```
n = 8
r = 3
result = nCr(n, r)
print(f"C({n},{r}) = {result}")
```

```
C(8,3) = 56
```

Question 4: Compute the Binomial Coefficient using Dynamic Programming

```
n = 10
r = 4
result = nCr(n, r)
print(f"C({n},{r}) = {result}")
```

```
C(10,4) = 210
```

Question 5: Determine the number of possible committees formed from a group

```
total_members = 12
committee_size = 5
result = nCr(total_members, committee_size)
print(f"Number of Committees = {result}")
```

```
Number of Committees = 792
```

Question 6: Calculate the combination value for given n and r

```
n = 15
r = 6
result = nCr(n, r)
print(f"C({n},{r}) = {result}")
```

```
C(15,6) = 5005
```

Question 7: A software company has 10 developers and wants to select 3 developers for a special AI project

```
total_developers = 10
developers_required = 3
result = nCr(total_developers, developers_required)
print(f"Number of Teams = {result}")
```

```
Number of Teams = 120
```

Question 8: A university must select 4 faculty members from a pool of 12 professors

```
total_professors = 12
committee_size = 4
result = nCr(total_professors, committee_size)
print(f"Number of Committees = {result}")
```

```
Number of Committees = 495
```

Question 9: A cricket academy has 15 players and needs to choose 5 players

```
total_players = 15
players_required = 5
result = nCr(total_players, players_required)
print(f"Number of Selections = {result}")
```

```
Number of Selections = 3003
```

Question 10: A company wants to select 6 customers from 20 volunteers

```
total_volunteers = 20
group_size = 6
result = nCr(total_volunteers, group_size)
print(f"Number of Groups = {result}")
```

```
Number of Groups = 38760
```

Question 11: A cloud provider has 8 servers and must allocate 3 servers

```
total_servers = 8
servers_selected = 3
result = nCr(total_servers, servers_selected)
print(f"Number of Combinations = {result}")
```

```
Number of Combinations = 56
```

Assembly Line Scheduling Solutions

Question 1: Find the minimum time required to manufacture a product using two assembly lines

```
entry_times = [10, 12]
exit_times = [18, 7]
line_times = [
    [4, 5, 3, 2], # Line 1 times
    [2, 10, 1, 4]  # Line 2 times
]
transfer_times = [
```

```
        [7, 4, 5], # Transfer from line 1 to line 2 (before station 1, 2, 3)
        [9, 2, 8]  # Transfer from line 2 to line 1 (before station 1, 2, 3)
    ]

    # Note: The problem description lists Transfer1 and Transfer2 in a way that sug
    # Transfer1 is for switching to Line 1 and Transfer2 for switching to Line 2.
    # However, standard assembly line scheduling problems usually define t[i][j]
    # as the time to transfer from line i to the *other* line before station j+1.
    # My function expects t[0] = transfer from line 0 to line 1,
    # and t[1] = transfer from line 1 to line 0.
    # Adjusting input for my function based on common interpretation:
    # t[0] would be transfer from line_times[0] to line_times[1]
    # t[1] would be transfer from line_times[1] to line_times[0]
    # Given 'Transfer1' [7,4,5] and 'Transfer2' [9,2,8] in the problem.
    # Assuming Transfer1 means time to transfer *to* Line 1, and Transfer2 *to* Lir
    # Let's map problem input to function parameters `a`, `t`, `e`, `x`
    # a = line_times, e = entry_times, x = exit_times
    # For t, problem gives Transfer1, Transfer2. Let's assume Transfer1 in problem
    # and Transfer2 corresponds to t[1] for current line 1 to other line 0.

    # The problem states Transfer1 and Transfer2 as lists of length N-1, where N is
    # These values represent transfer *between* stations. The interpretation used i
    # expects `t[line_from][station_index]` to be the time to transfer from `line_f

    # Let's align with the problem's example output, it implies:
    # Transfer from Line1 to Line2 are `transfer_times[0]` (problem's Transfer1, as
    # Transfer from Line2 to Line1 are `transfer_times[1]` (problem's Transfer2, as

    # Re-mapping based on my function's expected `t` parameter:
    # a = line_times (processing times at stations for each line)
    # t = transfer_times (transfer times between lines)
    # e = entry_times (entry times to each line)
    # x = exit_times (exit times from each line)

    min_time = assembly_line_scheduling(line_times, transfer_times, entry_times, ex
    print(f"Minimum Production Time = {min_time}")
```

```
Minimum Production Time = 35
```

Question 2: Determine the optimal assembly line path with minimum processing time

```
entry_times = [8, 10]
exit_times = [12, 15]
line_times = [
    [5, 6, 4],
    [7, 3, 5]
]
transfer_times = [
```

```
    [2, 1],
    [3, 2]
]
min_time = assembly_line_scheduling(line_times, transfer_times, entry_times, ex
print(f"Minimum Production Time = {min_time}")
```

```
Minimum Production Time = 35
```

Question 3: Compute the fastest route through assembly stations

```
entry_times = [5, 6]
exit_times = [4, 5]
line_times = [
    [3, 4, 5],
    [4, 3, 6]
]
transfer_times = [
    [2, 1],
    [1, 2]
]
min_time = assembly_line_scheduling(line_times, transfer_times, entry_times, ex
print(f"Minimum Production Time = {min_time}")
```

```
Minimum Production Time = 21
```

Question 4: Identify the assembly line schedule that minimizes total manufacturing time

```
entry_times = [7, 9]
exit_times = [8, 6]
line_times = [
    [4, 3, 5, 2],
    [5, 2, 4, 3]
]
transfer_times = [
    [1, 2, 1],
    [2, 1, 2]
]
min_time = assembly_line_scheduling(line_times, transfer_times, entry_times, ex
print(f"Minimum Production Time = {min_time}")
```

```
Minimum Production Time = 27
```

Question 5: Determine the minimum cost path through an assembly system

```
entry_times = [6, 7]
exit_times = [5, 4]
line_times = [
    [2, 4, 3],
    [3, 2, 5]
]
transfer_times = [
    [1, 2],
    [2, 1]
]
min_time = assembly_line_scheduling(line_times, transfer_times, entry_times, ex
print(f"Minimum Production Time = {min_time}")
```

```
Minimum Production Time = 20
```

Question 6: An automobile manufacturer has two assembly lines for producing cars

```
entry_times = [10, 12]
exit_times = [18, 7]
line_times = [
    [4, 5, 3, 2],
    [2, 10, 1, 4]
]
transfer_times = [
    [7, 4, 5],
    [9, 2, 8]
]
min_time = assembly_line_scheduling(line_times, transfer_times, entry_times, ex
print(f"Minimum Production Time = {min_time}")
```

```
Minimum Production Time = 35
```

Question 7: A smartphone company assembles devices using two production lines

```
entry_times = [8, 10]
exit_times = [12, 15]
line_times = [
    [5, 6, 4],
    [7, 3, 5]
]
transfer_times = [
    [2, 1],
    [3, 2]
```

```
]
min_time = assembly_line_scheduling(line_times, transfer_times, entry_times, ex
print(f"Minimum Production Time = {min_time}")
```

```
Minimum Production Time = 35
```

Question 8: A food processing company uses two packaging lines

```
entry_times = [5, 6]
exit_times = [4, 5]
line_times = [
    [3, 4, 5],
    [4, 3, 6]
]
transfer_times = [
    [2, 1],
    [1, 2]
]
min_time = assembly_line_scheduling(line_times, transfer_times, entry_times, ex
print(f"Minimum Production Time = {min_time}")
```

```
Minimum Production Time = 21
```

Question 9: Medicine bottles pass through filling, capping, labeling, and packaging stations

```
entry_times = [7, 9]
exit_times = [8, 6]
line_times = [
    [4, 3, 5, 2],
    [5, 2, 4, 3]
]
transfer_times = [
    [1, 2, 1],
    [2, 1, 2]
]
min_time = assembly_line_scheduling(line_times, transfer_times, entry_times, ex
print(f"Minimum Production Time = {min_time}")
```

```
Minimum Production Time = 27
```

Question 10: A circuit board manufacturing facility uses two parallel assembly lines

Gemini

```
entry_times = [6, 7]
exit_times = [5, 4]
line_times = [
    [2, 4, 3],
    [3, 2, 5]
]
transfer_times = [
    [1, 2],
    [2, 1]
]
min_time = assembly_line_scheduling(line_times, transfer_times, entry_times, exi
print(f"Minimum Production Time = {min_time}")
```

```
Minimum Production Time = 20
```